

MINGGU 2 — UNINFORMED SEARCH STRATEGIES

A.1 Tujuan Praktikum

- Mahasiswa mampu merepresentasikan suatu masalah pencarian sebagai graph (state space) dalam bahasa Python.
- Mahasiswa mampu mengimplementasikan algoritma Breadth-First Search (BFS), Depth-First Search (DFS), dan Uniform-Cost Search (UCS) dari nol (tanpa library pencarian siap pakai).
- Mahasiswa mampu membandingkan performa (jumlah node yang dieksplorasi, panjang lintasan, dan total biaya) antar algoritma uninformed search pada kasus yang sama.

A.2 Dasar Teori Singkat

Berdasarkan AIMA Bab 3.3–3.4, algoritma pencarian bekerja dengan membangun search tree di atas state-space graph. Setiap node menyimpan state, parent, action, dan path-cost. Frontier adalah kumpulan node yang sudah dibangkitkan tetapi belum diekspansi, dan strategi pencarian dibedakan terutama oleh struktur data yang digunakan untuk menyimpan frontier tersebut:

Algoritma	Struktur Data Frontier	Sifat Utama
Breadth-First Search (BFS)	FIFO Queue	Mengeksplorasi berdasarkan kedalaman terendah terlebih dahulu; complete dan optimal hanya jika semua biaya aksi sama
Depth-First Search (DFS)	LIFO Queue (Stack)	Mengeksplorasi node terdalam terlebih dahulu; hemat memori namun tidak complete pada ruang tak berhingga dan tidak optimal
Uniform-Cost Search (UCS)	Priority Queue (berdasarkan $g(n)$)	Selalu mengekspansi node berbiaya kumulatif terendah; complete dan cost-optimal untuk biaya aksi positif

Ketiga algoritma di atas termasuk kelompok uninformed (blind) search karena tidak menggunakan informasi tambahan mengenai seberapa dekat suatu state dengan tujuan — berbeda dengan informed search yang akan dipelajari pada Minggu 3.

A.3 Alat & Bahan

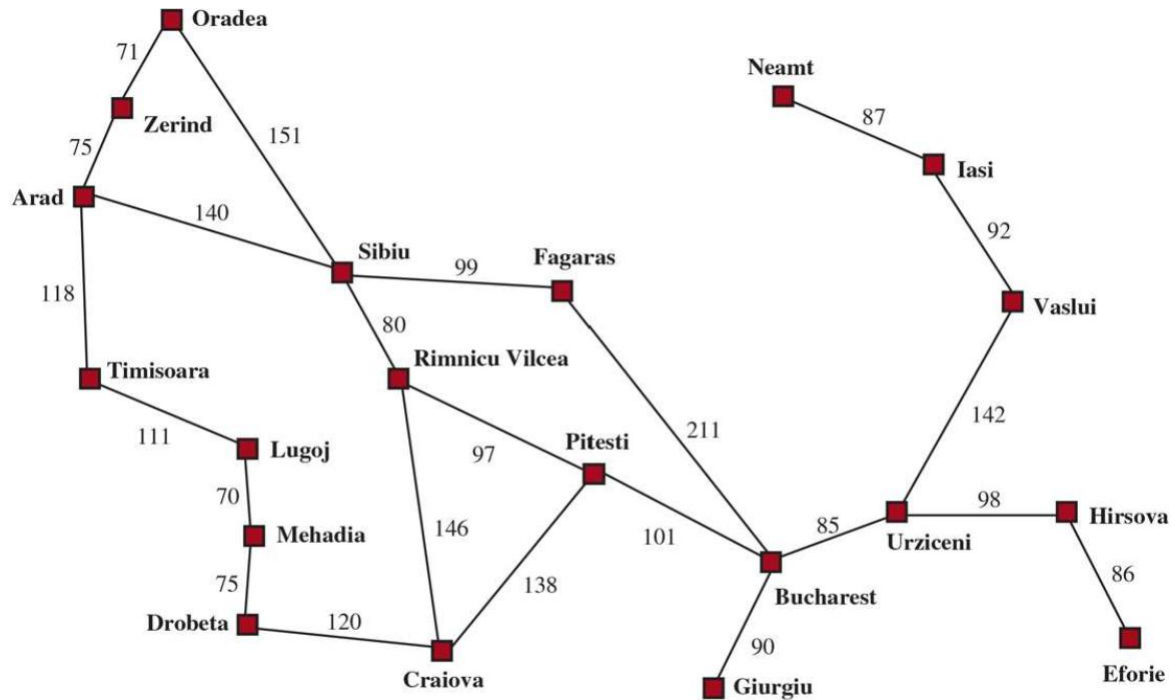
- Python 3.8 ke atas
- Jupyter Notebook atau Google Colab
- Modul bawaan Python: collections.deque dan heapq (tidak memerlukan instalasi library tambahan)

A.4 Langkah Percobaan

Percobaan 1 — Merepresentasikan Peta sebagai Graph

Kita menggunakan studi kasus peta Rumania yang menjadi contoh baku pada AIMA Bab 3: mencari rute dari kota Arad menuju Bucharest. Setiap kota direpresentasikan sebagai key pada dictionary, dan setiap tetangganya beserta biaya (jarak dalam km) direpresentasikan sebagai dictionary di dalamnya (adjacency list berbobot).

Kode Program 1 — Representasi Graph



A simplified road map of part of Romania, with road distances in miles.

```
import heapq
from collections import deque

# Graph peta Rumania (AIMA Fig. 3.1)
# Setiap key adalah kota, value adalah dict {tetangga: biaya aksi}
graph = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99,
              'RimnicuVilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'RimnicuVilcea': 146, 'Pitesti': 138},
    'RimnicuVilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'RimnicuVilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90,
                  'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87},
}

def reconstruct(parent, goal):
    """Menelusuri kembali dictionary parent untuk membentuk lintasan solusi."""
    path = [goal]
    while path[-1] in parent:
        path.append(parent[path[-1]])
```

```
return list(reversed(path))
```

Percobaan 2 — Breadth-First Search (BFS)

BFS mengeksplorasi frontier menggunakan FIFO queue (`collections.deque`), sehingga node yang lebih dulu masuk ke frontier akan lebih dulu diekspansi — hasilnya, BFS selalu menemukan lintasan dengan jumlah langkah (hop) paling sedikit terlebih dahulu.

Kode Program 2 — Breadth-First Search

```
def bfs(graph, start, goal):
    frontier = deque([start])    # FIFO queue -> ciri khas BFS
    parent = {}
    reached = {start}
    expanded = 0
    while frontier:
        node = frontier.popleft()
        expanded += 1
        if node == goal:
            return reconstruct(parent, goal), expanded
        for neighbor in graph.get(node, {}):
            if neighbor not in reached:
                reached.add(neighbor)
                parent[neighbor] = node
                frontier.append(neighbor)
    return None, expanded

path, expanded = bfs(graph, 'Arad', 'Bucharest')
print("Lintasan BFS :", " -> ".join(path))
print("Node dieksplorasi:", expanded)
```

Output yang Diharapkan:

```
Lintasan BFS : Arad -> Sibiu -> Fagaras -> Bucharest
Node dieksplorasi: 9
```

Percobaan 3 — Depth-First Search (DFS)

DFS menggunakan list Python sebagai stack (LIFO): method `pop()` tanpa index mengambil elemen terakhir yang dimasukkan, sehingga pencarian menyelami satu cabang sedalam mungkin sebelum mundur (backtrack).

Kode Program 3 — Depth-First Search

```
def dfs(graph, start, goal):
    frontier = [start]          # digunakan sebagai stack (LIFO) -> ciri khas DFS
    parent = {}
    reached = {start}
    expanded = 0
    while frontier:
        node = frontier.pop()    # pop() dari akhir list = LIFO
        expanded += 1
        if node == goal:
            return reconstruct(parent, goal), expanded
        for neighbor in graph.get(node, {}):
            if neighbor not in reached:
                reached.add(neighbor)
                parent[neighbor] = node
```

```

        frontier.append(neighbor)
    return None, expanded

path, expanded = dfs(graph, 'Arad', 'Bucharest')
print("Lintasan DFS :", " -> ".join(path))
print("Node dieksplorasi:", expanded)

```

Output yang Diharapkan:

```

Lintasan DFS : Arad -> Timisoara -> Lugoj -> Mehadia -> Drobeta -> Craiova -> Pitesti -> Bucharest
Node dieksplorasi: 8

```

Perhatikan bahwa lintasan yang ditemukan DFS jauh lebih panjang (7 langkah) dibandingkan BFS (3 langkah), meskipun jumlah node yang dieksplorasi lebih sedikit — DFS tidak menjamin lintasan pendek maupun murah.

Percobaan 4 — Uniform-Cost Search (UCS)

UCS menggunakan priority queue (modul `heapq`) yang diurutkan berdasarkan path cost $g(n)$, bukan urutan masuk. Baris `if cost > best_cost.get(node, float('inf')): continue` diperlukan karena `heapq` tidak mendukung penghapusan entri lama secara langsung, sehingga entri usang (lazy deletion) cukup dilewati saat muncul di frontier.

Kode Program 4 — Uniform-Cost Search

```

def ucs(graph, start, goal):
    frontier = [(0, start)]    # priority queue, diurutkan oleh path cost g(n)
    parent = {}
    best_cost = {start: 0}
    expanded = 0
    while frontier:
        cost, node = heapq.heappop(frontier)
        if cost > best_cost.get(node, float('inf')):
            continue           # lewati entri usang (lintasan lama yg lebih mahal)
        expanded += 1
        if node == goal:
            return reconstruct(parent, goal), cost, expanded
        for neighbor, step_cost in graph.get(node, {}).items():
            new_cost = cost + step_cost
            if new_cost < best_cost.get(neighbor, float('inf')):
                best_cost[neighbor] = new_cost
                parent[neighbor] = node
                heapq.heappush(frontier, (new_cost, neighbor))
    return None, None, expanded

path, cost, expanded = ucs(graph, 'Arad', 'Bucharest')
print("Lintasan UCS :", " -> ".join(path))
print("Total biaya :", cost)
print("Node dieksplorasi:", expanded)

```

Output yang Diharapkan:

```

Lintasan UCS : Arad -> Sibiu -> RimnicuVilcea -> Pitesti -> Bucharest
Total biaya : 418
Node dieksplorasi: 13

```

UCS menemukan lintasan dengan total biaya 418 km — inilah lintasan optimal (cost-optimal) untuk kasus Arad menuju Bucharest, berbeda dari lintasan yang ditemukan BFS maupun DFS.

A.5 Tabel Hasil Pengamatan

Lengkapi tabel berikut berdasarkan hasil percobaan Anda (gunakan hasil Percobaan 2–4 untuk baris pertama, lalu ulangi untuk pasangan kota lain pada Eksperimen Mandiri nomor 1).

Algoritma	Kasus	Lintasan Ditemukan	Total Biaya	Node Dieksplorasi
BFS	Arad→Bucharest			
DFS	Arad→Bucharest			
UCS	Arad→Bucharest			
BFS	____→____			
DFS	____→____			
UCS	____→____			

A.6 Eksperimen Mandiri

1. Jalankan BFS, DFS, dan UCS untuk pasangan kota lain, misalnya Timisoara → Bucharest dan Oradea → Eforie. Catat hasilnya pada tabel pengamatan A.5.
2. Modifikasi graph di atas menjadi grid world 5x5 sederhana (gunakan koordinat (baris, kolom) sebagai state dengan 4 aksi: atas/bawah/kiri/kanan, masing-masing berbiaya 1) dan tambahkan minimal 3 sel obstacle yang tidak boleh dilalui. Jalankan BFS dan DFS dari sudut kiri-atas menuju sudut kanan-bawah, lalu bandingkan lintasan yang ditemukan keduanya.
3. Tambahkan penghitung waktu eksekusi menggunakan modul time (time.perf_counter()) pada masing-masing fungsi pencarian, lalu bandingkan waktu eksekusi BFS, DFS, dan UCS pada peta Rumania.
4. (Tantangan) Implementasikan Iterative Deepening Search (IDS) dengan memanggil Depth-Limited Search berulang kali menggunakan limit kedalaman yang bertambah 0, 1, 2, ... hingga solusi ditemukan. Bandingkan jumlah node yang dieksplorasi IDS dengan BFS pada kasus Arad → Bucharest.

A.7 Pertanyaan Analisis

1. Apakah BFS pada percobaan ini menemukan lintasan dengan biaya termurah (cost-optimal)? Jelaskan berdasarkan hasil Percobaan 2 dan 4, mengingat peta Rumania memiliki bobot aksi yang berbeda-beda pada setiap sisi.

2. Mengapa DFS pada Percobaan 3 menemukan lintasan yang jauh lebih panjang (dalam jumlah kota yang dilewati) dibandingkan BFS?

3. Jelaskan mengapa UCS menghasilkan lintasan dengan total biaya paling rendah dibandingkan BFS dan DFS pada kasus peta berbobot ini.

4. Berdasarkan eksperimen grid world (Eksperimen Mandiri nomor 2), algoritma mana yang cenderung lebih hemat memori pada ruang keadaan yang besar? Kaitkan jawaban Anda dengan space complexity masing-masing algoritma ($O(b^d)$ untuk BFS/UCS vs $O(bm)$ untuk DFS).
